

Mid-Level System Design Interview

Learning Sources

[A Senior Engineer's Guide to the System Design Interview](#)
[System Design Mid-Level Cheat Sheet](#)

Notes

- Notes originally created in Jan 2025
- These note HAVE been reviewed by Claude Sonnet 4

Preface

- Candidates are often overwhelmed with system design. Thankfully, knowing even just the basics can go a long way.
- Often breadth of knowledge, over depth, is better for mid-level developers. Studying too deep in something can result in diminishing returns for interviews.
- 80% of system design interviews generally only involve 20% of concepts within broader system design
- "A Senior Engineer's Guide to the System Design Interview" article (notes based on it) was created via:
 1. Listening to 30+ hours of system design interviews and lessons
 2. Performing data analysis to ID 50+ of the site's (which is a site for interviews) highest rated interviewers
 3. Built interview guide using above, then peer reviewed with other engineers
- Guide is in four parts:
 1. How to Approach a System Design Interview
 2. Key Topics to Understand
 3. 3-Step Interview Framework
 4. Example System Design Problems

Part I: Intro to Sys Design

- Not all system design answers require you to build distributed systems, which can be much more complex.
- Even working at a company for a long time, you may not know great depth about a system's design or advanced system design concepts. Continued education, through reading books, docs, guides, and practice, thus is very important for proper system design education (and interviews), as such.

Engineering vs Design Problems

- Engineering Problem - when solving a problem with a lot of data and you're sure there is one best solution. May require many prototypes, but as long as final solution is top notch, solution can be repeated a million times

-Design Problem - when solving a problem with little precedent, and there is no or little fixed determined outcome for the solution, requiring all types of different ideas to be considered.

-Design problems require a lot of brainstorming and creativity. Try crazy stuff, improvise, and "build your way forward." Seek a beautiful aesthetic in the process.

-Best way to bomb sys design interview is to treat it like an eng design problem. The less code you write in sys design interview, the better. Whiteboard abstractions, instead.

-Collaborate with interviewer and try diff ideas.

Sys Design Interview vs Coding Interview

-Coding is "retrieving" a solution. Sys design is creating one. Coding == science, design == art. Sys design == creating lines and shapes for someone else to color in.

-Sys design has no correct answers, though some will be incorrect, and some more suitable than others.

-Pretend like you are a tech lead, describing how the system will work to a junior eng not involved in designing it. Expect and answer interviewer questions.

-Communication is key. Back-and-forth will be frequent. Interviewer could be less talkative or more collaborative, too, though.

-Complete mock interviews with a variety of interviewers before doing real interviews.

-Check out *interviewing.io* for paid mock interviews (and recorded interviews) with actual FB, Amazon, Google, etc. employees.

-Get your interviewers to ask you questions and leave breadcrumbs for them to do so. Get them to dig deeper into your solution.

-Flex what you do know, and speak less on what you don't

The Goal of the Interviewer

-To gather enough data to determine if to hire you. To determine if your knowledge and ability is sufficient for the position.

-You must show you know the fundamentals, end-to-end, and be able to name and explain, at least at a high level, each part of the system, trade-offs you make, and a final solution.

-Interviewer wants to see:

- a broad, base-level understanding of sys design fundamentals
- back-and-forth about problem constraints and parameters
- well reasoned, qualified decisions based on engineering trade-offs
- the unique direction your experience and decisions take them
- a holistic view of the system and it's users

-Interviewer is not looking for:

- deep expertise in the given problem domain

- assumptions about the problem
- specific answers with iron-clad certainty
- a predefined path from the problem start to end
- strictly technical considerations

Deep Domain Expertise Not Required

-A wide range of basic topic knowledge is required, while great depth into those knowledge is not.

-Ex. Basic Knowledge - Do know diff between SQL and NoSQL DB, their performance characteristics, and types of apps they might be useful for. Do not need to know the internals of each DB or how DB work at detailed level.

-Start first from clarification, collaboration, and understanding with interviewer, then move deeper into design.

Back-and-Forth

-Interviewers do expect a lot of back-and-forth about problem constraints and params. Do not assume requirements!

-Problem prompts will be light on detail. Clarify the type of service. What exactly does it do, in depth? What features? Type(s) of user? Expected traffic? What limits? Clarify models and purpose.

-For attributes, consider details. Public or private/protected? What requirements does ID need to fulfill to avoid collisions? Attr type, etc.

Not a Predefined Path

-A predefined path is not the end goal. Rather desired is your own unique direction, which comes from your own unique experience and education.

-Choose Your Own Adventure book. A multiple of sub-components, each which could lead to a design problem or solution of it's own.

-Sketch out the big picture first.

-Feel free to say when you don't have experience in something, like caching, but still think it would be a good solution, and why.

A Holistic View

-Interviewers seek a holistic view of the system and it's users. Breadth over depth! A complete system!

-Computer systems serve human users, so focus on user experience and how your tech designs enable them.

-Consider user related details in relation to system design. Ex. for image hosting site, do users need to login first, or can images be uploaded anon? No login gives quick use and low interaction. Login allows user galleries, more community features, and metadata gathering. What about both, with limited feature sets for no login users?

Green Flags, Red Flags, & Signposts

-Green Flag - flag showing things are going well, interviewer is engaging with you well, and impression is positive

-Red Flags - flag showing you are going astray and need to get interview/er back on track

-Green Flags:

- You are communicating honestly about what you know and don't
- The interview is properly collaborative, with questions, feedback and integration, and good educational tone to convo.
- Your role determines who sets the pace of the interview. Mid-level and lower should be interviewer determined. Interviewer asks clarifying questions. Interviewer may change requirements and ask how you handle the new ones. Senior level should be directing the interview themselves.
- You are taking time to collect thoughts and refine solutions before offering them out loud

-Red Flags:

- You are just, "playing the game, keeping talking, and making sure nothing explodes"
- You find yourself pushing back against interviewer feedback. They know a good answer. Follow it.
- You ignore problem prompts and skip too far ahead without addressing concerns first.
- Being overconfident, talking too much, and taking away too much interviewer control.
- You leave long periods of silence (up to minutes) in explanation. Ask for help & explain thoughts, instead!

-Signposts

- Seniors should not have awkward pauses, but for mid-level and lower, filling them in with trash is worse than silence. "Better to remain silent and be thought a fool than to speak out and remove all doubt."
- Make decisions. Don't say, "we could use X or Y, with pros and cons A and B," but then move on without making a choice, and explaining why, first.
- Interviewers want to ID "importers." Those who say, "I will use Kafka," but then can't describe what it is, or talk about why Kafka vs other options for queues. "I will use XYZ, due to [rationale]."
- Speak in generics, not specifics. "I will use a queue," not, "I will use Kafka."

Part II: 15 Fundamental Sys Design Concepts

Three Core Concepts

I) There is No Single Right Way to Design a System

-This section contains two videos of actual system design interviews, where ex-Google staff developers were asked: "Design [Interviewing.io](https://interviewing.io)," which is essentially three interviews in one, and requires "Design Google Docs, Design a Remote Compiler, and Design a Recording Service." Time limit, 30 mins or less. They design the system in very different ways, with different considerations and outcomes. See the interviews here:

[Two Ex-Google System Design Experts Compete: Who will design the better system? \[Part 1\]](#)

[Two Ex-Google System Design Experts Compete: Who will design the better system? \[Part 2\]](#)

II) General Rules of Thumb

-If an interviewer interrupts you, it's probably because you're going off track. Let them explain why and ask them clarifying questions, then get back on track.

-Prior experience affects both sides. The interviewer may ask you things you know, or challenge you with things you don't. If challenged, just come up with a reasonable solution, be honest about gaps, and show willingness to learn.

-If you know a little about your interviewer's background, prepare just for them. Same for company.

-Use time during interview to cover more, not go into deep depths. Make sure to finish the problem.

-Explain why you made the decisions you made. Why > what. Be sure you can talk on trade-offs.

-Keep it simple. Don't use distributed systems if you don't need them.

-Do plenty of mock interviews before real ones (interviewing.io).

III) Words to Say In Specific Scenarios

-Scenario: "I don't know what to do":

- Response: "I don't know, and I will look more into it after the interview, but my best guess is ..., and here is why ..."
- Response: "I don't have much experience with ..., but from reading up on it, I know ..."
- Response: "Honestly, I have never heard of that before. Could you tell me more about it?"

-Use a lot of team words, like "we," and "let's"

-Handwave (brush off or quickly move through) complex or undesired topics for the sake of time. "We'd have such-and-such, but that would take a lot of time to get into, so let's keep moving on with next-thing, for now."

-Pinpoint tricky aspects of problems right off the bat, instead of trying to avoid them. "The challenge which we would face would be ..., but let's finish... before coming back to that, and I may need more navigation with the implementation of this when we come back to it."

-Interviewers can be warm (chummy and collaborative) or cold (want you just to do it by yourself).

-Deal with cold interviewers by prompting them only where questions require them to get involved, if you're too far off track. "Correct me if I'm wrong, but I think I can do X because of Y." Do less thinking out loud with a cold vs warm. Prompt cold interviewers more. "I think we can do X because of Y. Do you agree?"

-The more questions you ask (past an expected limited amount), you weaker you can seem. Ask questions, but don't drown interviewers in them. Show you know stuff already and don't need confirmation on everything. Check in at milestones, only, not over each detail.

12 Fundamental Technical Sys Design Concepts

I) APIs

-Application Programming Interface - the set of rules and contracts through which two or more software entities communicate.

-Ex. A hot dog stand API that reads in customer orders, checks stock and cook time, then serves back order details.

-REST API - Representational State Transfer API. API is modeled based on the resources in a system. URIs are based on HTTP verbs (GET, PUT, POST, PATCH, DELETE) and perform actions on server side resources. Take requests. Produce responses.

-REST API Strengths - A structured way for getting and modifying DB data. Widely used and works in most situations. Tooling for generating API documentation widely available.

-REST API Weaknesses - Each entity in the DB must have its own requests written for it, unlike GraphQL, where single request query can gather many diff entities. Not as space efficient as RPC.

-RPC - Remote Procedure Call - Allows for execution of a procedure or command on a remote machine.

-RPC Strengths - More space efficient with compact binary protocols. Can be used for internal and external communication.

-RPC Weaknesses - Tighter coupling between services. Potential timing issues in unreliable networks.

-GraphQL - Can query multiple entities all at once.

-GraphQL Strengths - Good for customer facing web and mobile apps frequent UI changes, as easily makes all entities available to front-end devs w/o more back-end routes needing to be written.

-GraphQL Weaknesses - More up-front work required to set it up. Less friendly to external users than REST API, in which docs are auto-generated and well known HTTP methods form basis. Not good for systems which require data aggregation (grouping) on the back-end.

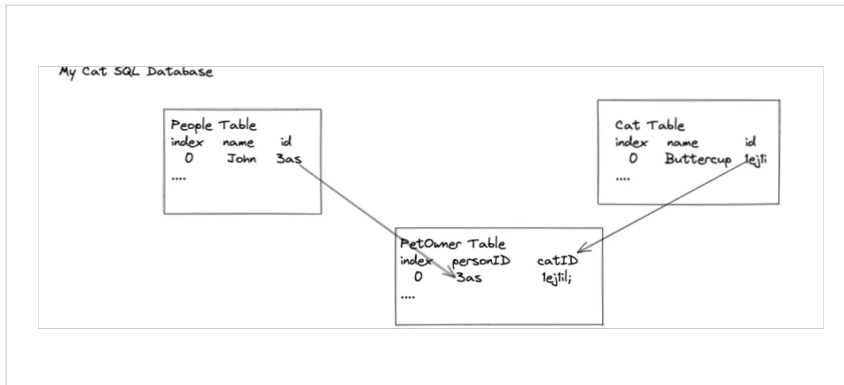
-Example Calls:

REST - `@app.post("/order/{orderNumber}/?query_params"){ ... }`

RPC - `/placeAnOrder(OrderDetails order)`

II) Databases

-SQL - Structured Query Language. SQL is used for relational databases, where data in one table relates to another table(s). An account has orders and links to them to a primary key via a foreign key, and so forth. Each table row is a data entry, and each column is a field within that entry.



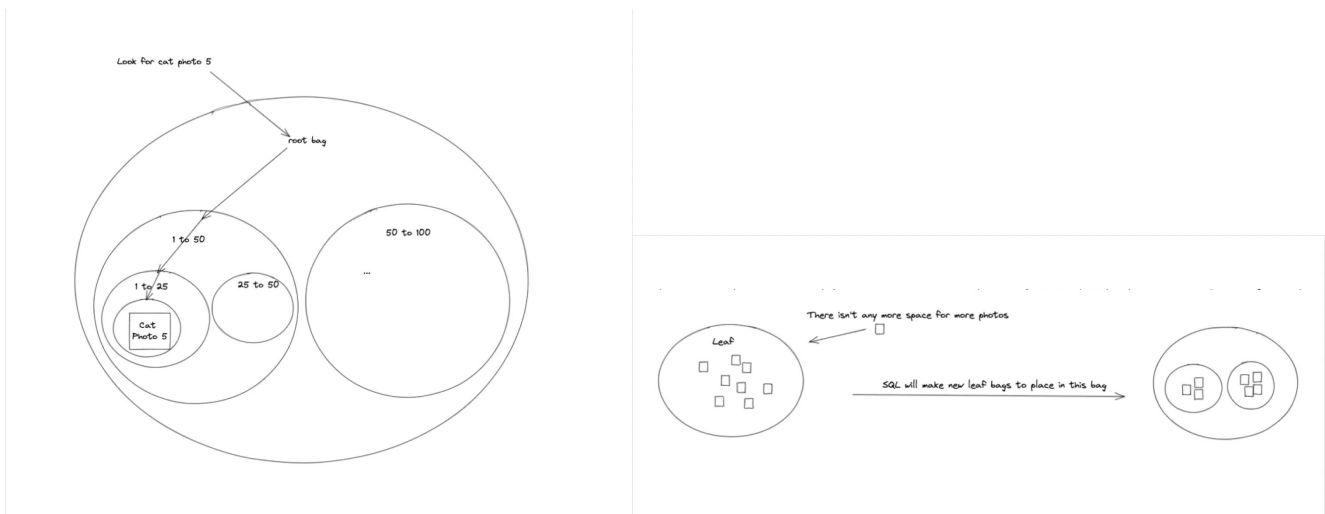
-SQL Strengths - SQL queries can be optimized in-depth. SQL has high ACID compliance (atomicity, consistency, isolation, durability).

-SQL ACID:

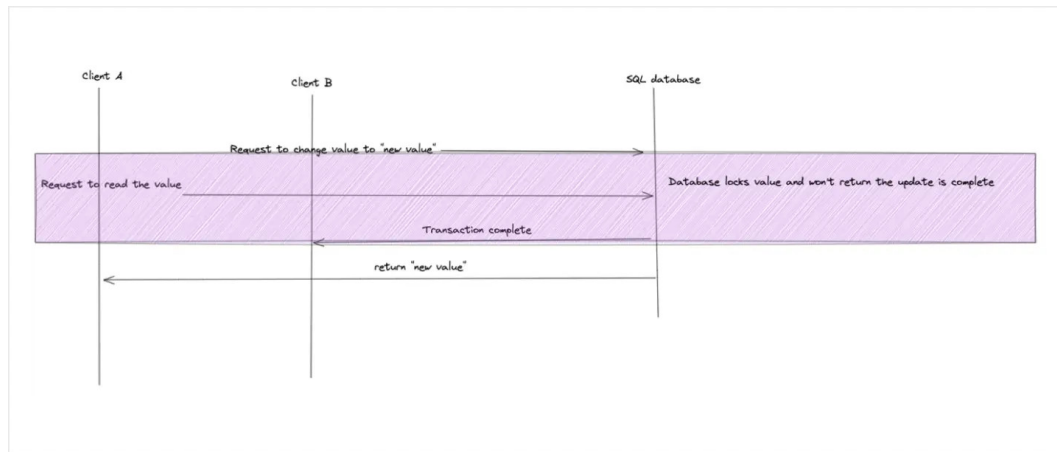
- Atomicity - a transaction completes or doesn't. No half writes.
- Consistency - Requests are forced to wait while other requests complete. There is no stale data.
- Isolation - All read and write transactions are not impacted by other transactions.
- Durability - If transaction fails, data is not lost and can be recovered

-SQL Weaknesses:

- Not as fast with binary trees as NoSQL (can be sped up via indexing). Slower for writing, in general, than NoSQL, as table data is stored in paginated, "pages," and whenever a write operation occurs, whole page with data must be re-written, which is slow.



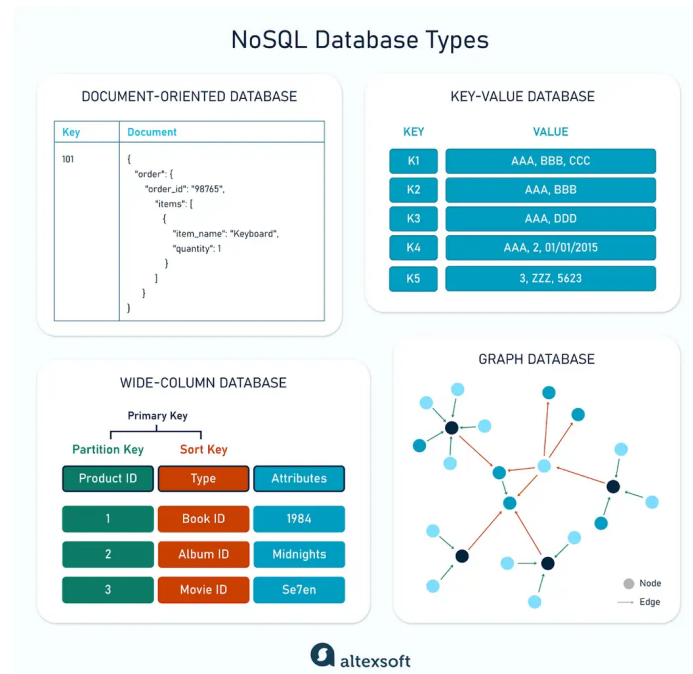
- Since SQL locks more when request is occurring, can result in being slower if many users all making same time requests.



- Mixed schema data also does not work best with SQL

-NoSQL - Data is stored in files which contain key/value pairs or other nested text syntax. Objects can nest other objects. All three tables of a SQL DB might live in a single text data file. Relation is not important, and each NoSQL object is more of an independent "card."

-Type of NoSQL DB:



- Key Value - Most popular type. Data values mapped to unique keys. Often uses JSON syntax. Data can be sharded (split) across nodes.
- Document DB - Similar to Key Value stores, but can also perform aggregate searches across data and store result in JSON, XML, YAML, etc.

- Columnar DB - Store info in tables but allow de-normalized data. Indexing column, instead of row based. Efficient when query involves computing same value types across multiple values.
- Graph DB: Store complicated node and edge relationships. Easy graph traversal and modification right out of the box

-NoSQL Strengths: Does not require rigid schema. Faster to write. Many, like DynamoDB and MongoDB include sharding and scaling out of the box. NoSQL is easy to scale, too.

-NoSQL Weaknesses - Slower to read for complex queries. Not good for when strong data consistency is required. Limited in the types of queries they can perform efficiently.

-In general NoSQL is faster for writes, but slower for reads:

Drivers	Lap 1	Lap 2	Lap 3
T.F	2:31	...	...
G.K	2:29	...	...
T.P	2:40	...	...

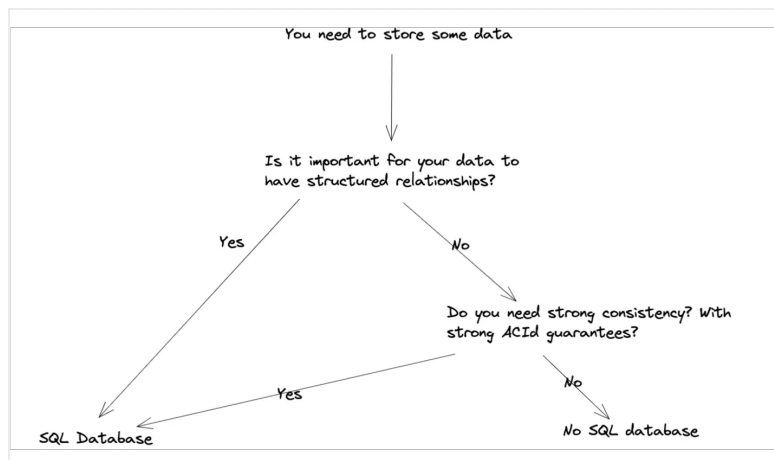
Having a table like this is easier to read but is too slow since you will need to search for the driver then the lap.

G.K	2:29
T.F	2:31
T.P	2:40
...	

Have a log list like this where you just add to the bottom is much faster for recording times. Sure its a little harder to read but we can deal with that later.

-SQL vs NoSQL:

- Use SQL when data needs structured relationships, high data consistency and ACID, or building more read than write heavy app.
- Use NoSQL when have frequently changing unstructured or semi-structured data, when need horizontal scalability for large vols of data, when more write than read heavy app, if high ACID and data consistency not as important, or when building distributed system.



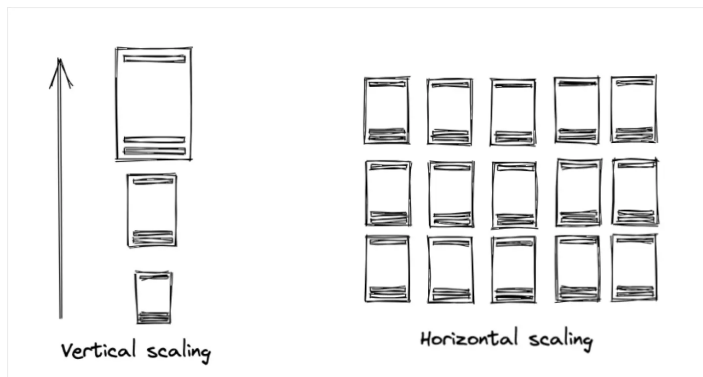
III) Scaling

-Questions on scaling to many unique users are common in interviews.

-Scaling is the steps taken to allow more users to visit your app and for the app to still be able to be performant with more users.

-Vertical Scaling – Making the computer which serves your app more powerful. Upgrade HDD, RAM, CPU, etc..

-Horizontal Scaling – Buying or renting more computers and distributing load across them. Computers communicate with each other via network. Primary forms are DB scaling, compute scaling, and hashing scaling.



-Vertical Scaling Strengths:

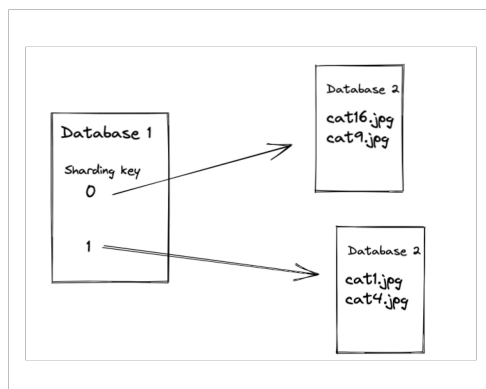
- Easier to scale initially in this manner as does not require changes to software architecture.
- Less latency as software communicates locally rather than across network.

-Vertical Scaling Weaknesses:

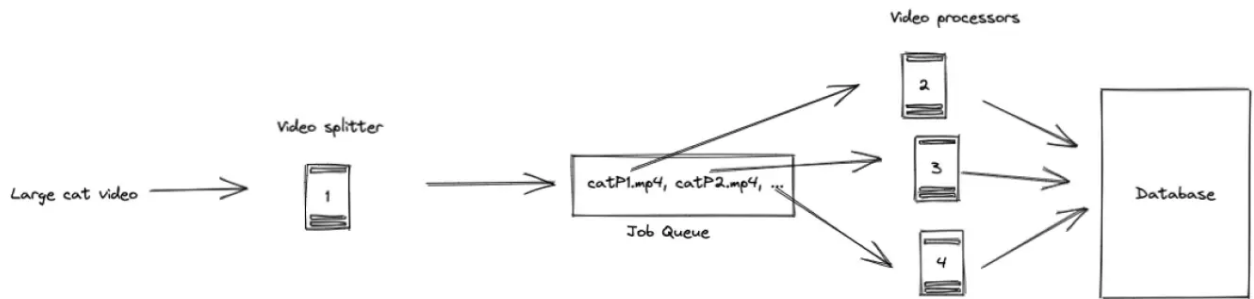
- Eventually physical limitations of upgrading a computer will prevent upgrading any further, or at least very expensive to do so
- Single point of failure, due to single server (machine)

-Two main types of horizontal scaling:

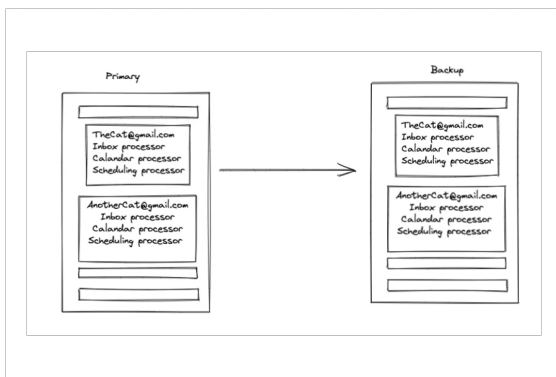
- Database Scaling – Adds DB replicates and shards (splits) large DB into smaller DB. Hash function determines which DB to query when query. Can shard as much or little as desired.



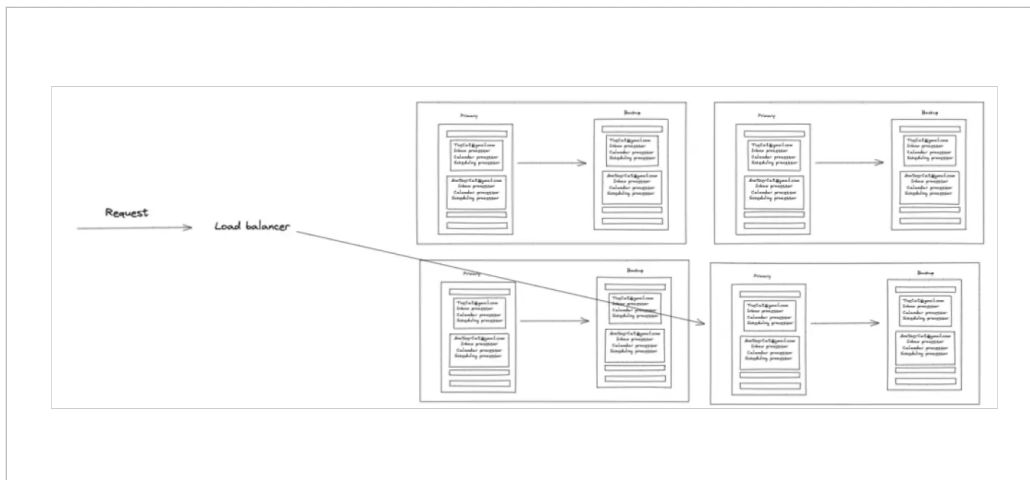
- Compute Scaling – Different computers work together in parallel to solve problem split into parts (ex. large video processed in sections by multi computers in parallel).



-Combining Horizontal and Vertical Scaling – Use horizontal for services and processes which need low latency calls to each other. Adjust each node, as required, to provide this. Use vertical for backups and being able to move users between 1-m nodes as system demand dictates.



-Ex. of combined scaling – Gmail, where many computationally inexpensive processes run in parallel for a user, and different nodes contain users of similar geographic location. Amount of power given to user-in-region node (server) is horizontal scaling. Vertical scaling comes into play as backup nodes and latency reduction duplicate nodes run and take total usage and latency numbers into account for scaling how many nodes are live.



IV) CAP Theorem

-Consistency, Availability, Partition tolerance. A fundamental theorem for distributed systems.

-Consistency – every node in the network will have access to the same data

-Availability – the system will always be available to users

-Partition Tolerance – the system will still work even in case of fault in network or communication

-CAP states that during a network partition (break-down in comm between nodes in a distributed system), the system must choose between consistency and availability - it cannot maintain both. When the network is functioning normally, all three properties can be satisfied.

-Partition (fault) tolerance is almost always necessary, so the choice comes between consistency and availability.

-For some systems, like financial consistency is the obvious choice. For others, like Tik Tok, availability becomes more important.

-Even if a system lacks immediate consistency, system can still have eventual consistency, on delayed CRUD, etc.

V) Auth & Basic Security

-Security is often a trade off between total safety (a wall) vs total convenience (a big hole in the wall). Security should minimize risk, while still allowing user functionality.

-Authentication – verifying the identity of a service's users

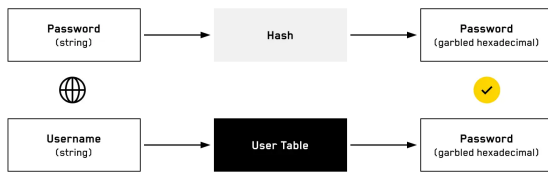
-Authorization – determining which permissions users have



-Storing passwords – Username and password combos. If do auth this way, make sure to hash passwords, and not store in DB plaintext, in case DB gets compromised.

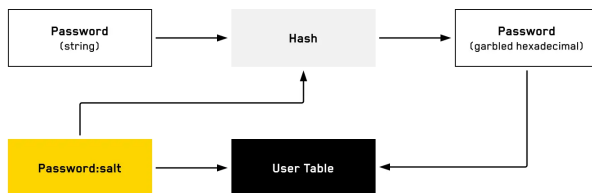
-Hash function – a function, generally cryptographic, that accepts input and maps it to a smaller value. While input value can determine output value, output value cannot determine input value. Same input, same output.

-Storing PW w/ hashing – When user signs up and creates password, run the password through a secure hashing function, then store output. When user logs in, hash their password again. If stored hashed password == login hashed password, password is correct.



-Salting – Adding characters to the password that are not actually part of the password. If password is compromised, thus hard to tell what are real vs fake parts of PW.

-Salting helps with Rainbow Table attacks, where attackers have massive lists of common passwords, where they can hash them all, using same algos any software does, and then if hash match, password match, password known. Then, if attacker gets DB access, they can compare their hashed passwords to your hashed passwords, to get user passwords. Ex. turn *password123* into *password123:a8h2rmd1tb* to prevent "know" hash being generated for pwd.



-Do not write your own hashing and salting functions. Use reliable libraries, using trustworthy hashing algos, instead.

-Web sign-in forms should always communicate via HTTPS to ensure credentials don't pass through the web in plaintext. All web traffic really should be sent via HTTPS, but especially credentials.

-Do not log passwords. Avoid logging POST bodies and GET params. Log URL paths and response codes, instead, and only fields when sure not passwords.

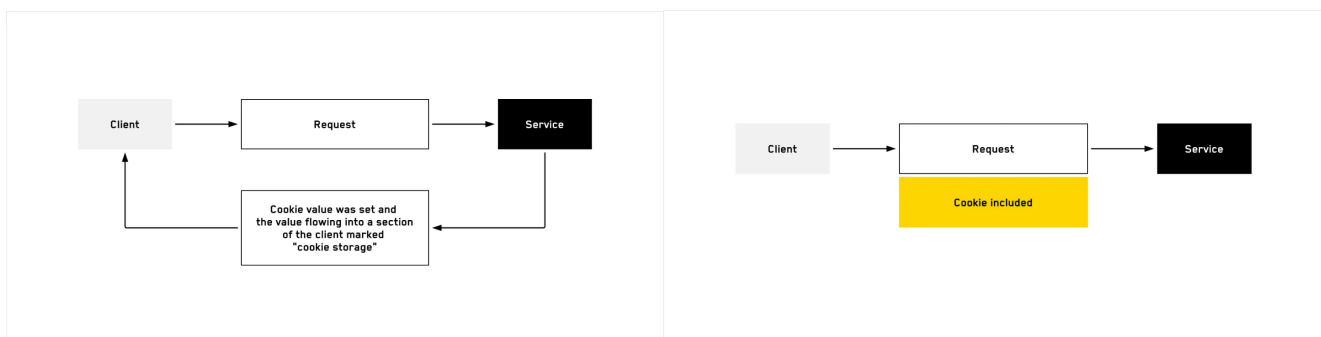
-After auth has been verified, need to provide user a way to prove they are authenticated for rest of session. Multiple ways to do this:

- Session Tokens – A randomly generated token used to show auth for that session. Can store in a cookie for subsequent requests. Session token should also be stored in hashed form. Session token should expire, and this can even happen mid session, without user being aware of. The faster the expiration, the less time for an attack on it (but more re-auth required, too). Encrypt all traffic via HTTPS, to also encrypt tokens.

- JWT – JSON Web Tokens. Complex tokens, in JSON form, which can store username, email, authorization, and authentication permissions, etc.. Token is validated as expected token by attaching a signature from the private key which verifies the token's legitimacy. Token itself is transparent and can be used across multiple services.

```
ex. {
  sub: 'user123',           // token subject
  email: 'user@domain.com', // identifier
  iat: 1665385660,         // issued at timestamp
}
```

-Cookies allow you to store your session token and JWTs client side, so they can be sent back to server on each request. Typically done via browser cookies. A cookie is a simple text string, which the browser sends back to server with each request (potentially limited by path). Server may also send *Set-Cookie* header with responses, to set a cookie on user browser, as well as optional cookie expiration.



-Cookies should only be accessible by service that sets it. Setting *Secure* flag on cookie tells browser to only send in HTTPS requests. *HttpOnly* flag says to not allow JS to access cookie. *SameSite* flag tells browser to only allow site that created it to access cookie (prevents cross-site request forgery (CSRF) attacks).

-Web Auth and Basic Security Summary:

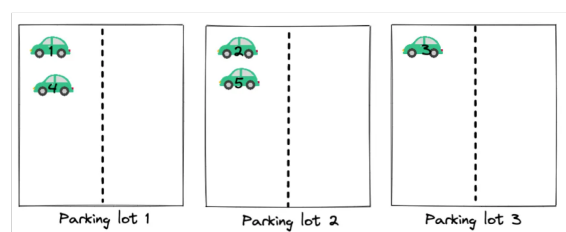
1. User signs up. We salt and hash their password and store that value.
2. User logs in with username and password. We verify by hashing pwd (with their salt included).
3. We send them a session token, JWT, or other token and store in cookie
4. On subsequent requests, browser sends back token cookie, which we use to re-verify auth/authn.
5. Periodically, session token or JWT expires, and new one sent to client via new cookie header
6. If login session expires, we repeat from step #2

VI) Load Balancers

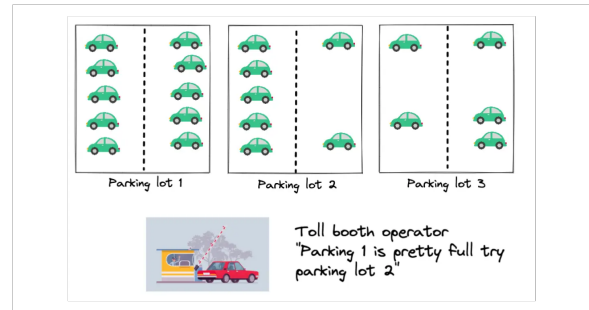
-Load balancers help evenly distribute traffic to distributed systems. Machines are added as load increases or failures occur.

-Options for load balancers

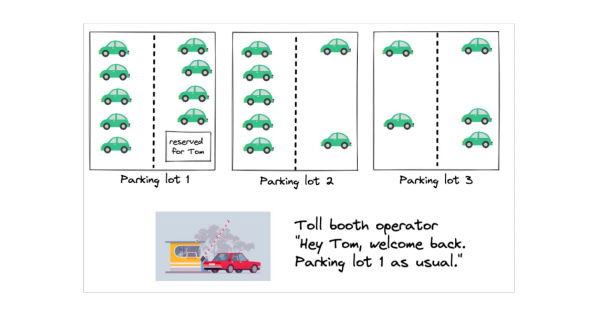
- Round Robin – If n systems, load balancer will send requests to them in order (instance 01 → instance 02 → instance 03 → instance 01 → ...). Mostly works well, but if one machine is processing requests slower, it can be overloaded.



- Least Connections / Response Time – Requests are sent to server that currently has least amount of connections or minimum response time. Useful for apps where response time can vary, but each server has about same resources allocated to it.



- Hashing – Hash function returns a server to send request to based on directives of hash function. Ex. Send to server closest to them.



VII) Caching

-Caching – storing an expensive computation result or asset so you don't have to compute it or fetch it again. Reduces latency of expensive network computations, network calls, DB queries, and asset fetching.

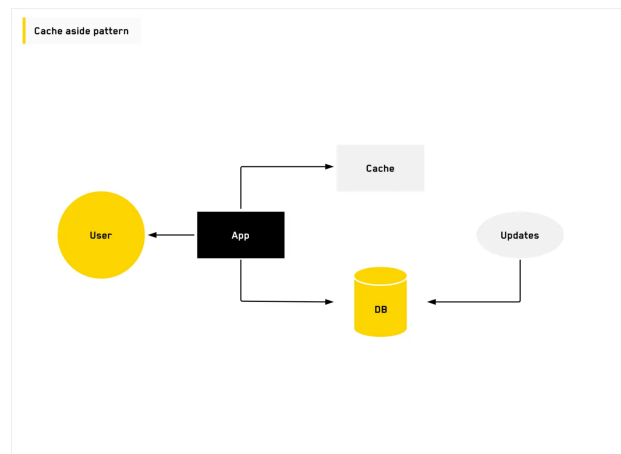
-Common in read heavy systems, like Twitter and Youtube (cache most popular videos, posts, etc.)

-80/20 Rule – Store 80% of read requests in 20% of storage (or memory). The popular data is stored and returned quickly.

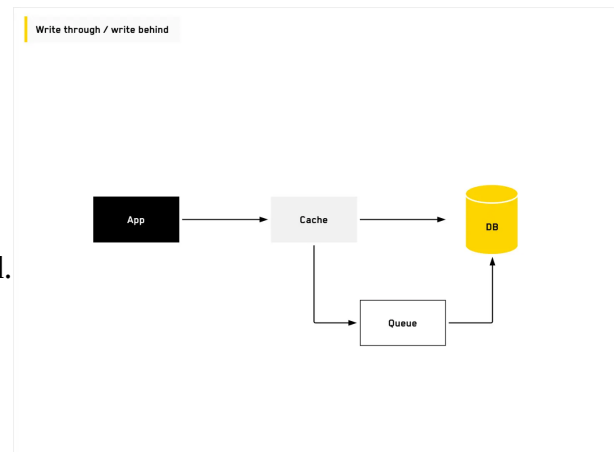
-Content delivery network (CDN) - specialized caching, often used for media, which distributes assets according to geo region of user.

-Popular Caching Patterns:

- Cache-aside pattern – Try and get data from cache. If not in cache, get from DB, put into cache, then return data to user.
 - Advantage: only caches what is read.
 - Disadvantage: cache can become stale if lots of DB updates.
 - Disadvantage: If a lot of calls for data not in cache, caching before returning can slow down app. If too many of these “cache misses,” move to another pattern.



- Write-through & Write-back – App writes data to cache, then the cache (sync or async) writes to DB. If sync, then “write-through.” If async, then “write-back.” Useful for if slow DB writes.
 - Disadvantage: Writing all data to cache could overload cache, which might not even be read. Also, if DB goes down before data written to DB, can cause data inconsistency.



-Can cache data on client side, such as thumbnails. Then, data can be re-served from cache (or local memory), instead of having to send through network again.

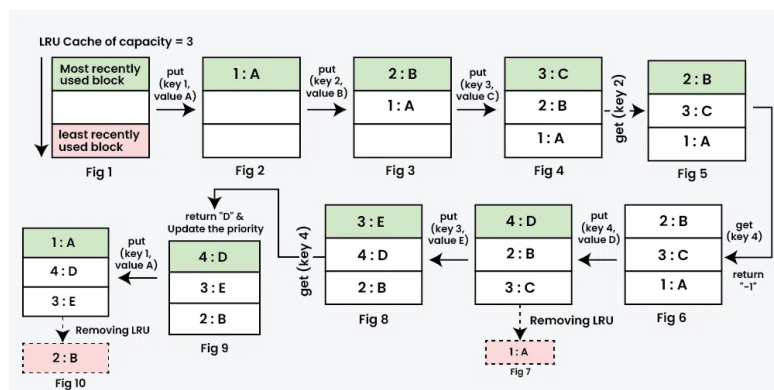
-Cache Invalidation - Expiring or invalidating cache after a certain period time, to prevent cache from getting stale. Cache can become stale if a lot of DB updates occur since cache created. discuss this during interviews.

-LRU Cache Invalidation - Least recently used. Caching strategy for dealing with limited cache. Order each item put into cache by "most recently accessed", and update order each time use that item from cache. Then, when cache full, clear the item not used for the longest time.

-LRU Example:

1. Put A into cache and timestamp 1
2. Put B into cache and timestamp 2
3. Put C into cache and timestamp 3
4. Read B from cache and update B timestamp to 4
5. Put D into cache and timestamp 5
6. Try to put E into cache. Cache is full. Remove A, as it has oldest (LRU) timestamp (1) and insert E into spot where A was w/ timestamp 6.

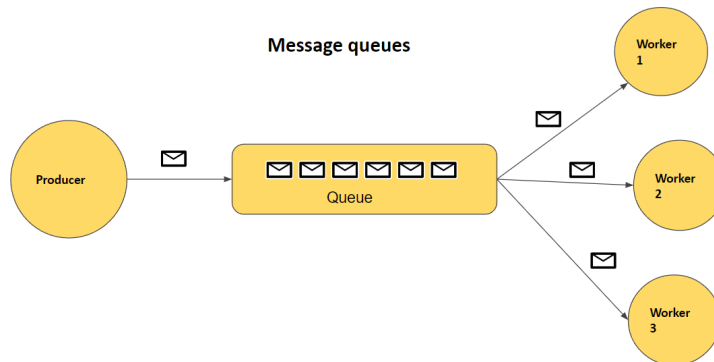
-LRU Example II:



-As 80% of data reads come from 20% of data, LRU cache invalidation works well as it stores what is being read, and removes what is not being read, which in turn returns request latency for cache.

VIII) Message Queues

-Queues make distributed systems more efficient, as they prevent systems from becoming overwhelmed by too many requests, while still eventually fulfilling all requests.



-Async Systems - In which client sends system a message, but instead of being immediately processed, is instead sent to message queue or broker, then eventually picked up from queue for processing. "Fire and forget" calls.

-Async Systems Strengths:

- Useful for systems making a lot of DB updates, to not overwhelm (and potentially take down) DB
- Useful for system which has expensive code, which may take a long time to finish
- Can be used to deliver messages to multiple systems, instead of client have to send request to all systems
- Queues decouple client from server by eliminating need to know server address, as can just send to queue

-Messaging queues work via Producers and consumers/subscribers, similar to an Observer pattern.

-A queue message is data that flows through the async system. No particular data model is enforced

-Producers are system which produce messages. The messages are posted to queue. Consumers then detect queue messages posted, and pick up message from queue, for processing.

-Message / Topic Queue is the link between producers and consumers. Producers publish to this and the queue delivers the published messages to consumers.

-Common Message Queues:

- Amazon SQS
 - Type: Simple managed queue service (AWS)
 - Pattern: Point-to-point messaging (one message, one consumer (consumer can have multiple instances))
 - Use Cases: Task distribution, workload decoupling, microservice communication
 - Strengths: Fully managed, easy to use, built-in AWS integration
 - Limitations: No pub/sub pattern natively, polling-based consumption, 14-day message retention maximum

- Apache Kafka
 - Type: Distributed event streaming platform (captures, publishes, and delivers real-time data streams, enabling applications to publish and subscribe to continuous flows of events)
 - Pattern: Pub/sub with consumer groups and topic partitioning
 - Use Cases: Event sourcing, log aggregation, metrics, activity tracking, real-time analytics
 - Strengths: High throughput, long-term retention, stream processing, exactly-once delivery
 - Limitations: Operational complexity, higher learning curve, requires infrastructure management
- RabbitMQ
 - Type: Traditional message broker
 - Pattern: Multiple exchange types (direct, fanout, topic, headers)
 - Use Cases: Complex routing requirements, RPC patterns, legacy system integration
 - Strengths: Flexible routing patterns, mature protocol support (AMQP), easy deployment
 - Limitations: Lower throughput than Kafka, more infrastructure management than SQS

-SQS simplest to operate, Kafka best for fast event streaming, RabbitMQ for most flexible routing options.

-Ex.

1. Customer orders a product online. Order processing service (Producer) gets order and creates message.
2. Web pushes order message to queue (ex. Kafka, RabbitMQ), then sends customer to "thank you" page.
3. Message waits until queue until consumers are ready.
4. Various consumers pull message and act accordingly. Inventory service pulls and updates stock levels. Payment Processing service pulls and charges credit card. Shipping service pulls and generates shipping label. Customer notification service pulls and sends confirmation email.

-Ex. II

1. ActiveCampaign customer updates account products (account "package").
2. Billing team Provisioning service (Producer) updates account package, then publishes "accountUpdated" message (containing update details) to AWS SNS topic (which can fan message to multiple destinations, adding easy extensibility)
3. SNS sends message to AWS SQS queue (message queue and broker), which handles message workload distribution across multiple consumers and has built in dead-letter queue.
4. AWS Kafka Connect SQS Connector (SQS Consumer) polls the SQS queue, gets message, then publishes to Kafka topic.
5. Other teams (Kafka Consumers) subscribe to Kafka topic to be notified of account package updated.

-Message Queue Strengths:

- Guaranteed delivery
- No duplicate messages delivered

- Order of messages is maintained
- At least once delivery with idempotent consumers

-Kafka and RabbitMQ are the most commonly used message queues, but remember to not talk about specific brand tech interviews unless you have enough experience to talk about them and alternative products, as well.

IX) Indexing

-A mechanism by which underlying DB data is mapped for faster retrieval

-Typically data retrieval works via fetching a block of data from hard disk and storing in primary memory, then looking up desired data in ram, and repeating this until desired data is found

-DB Indexing - speeds up data fetching by creating a key-value index for data, where key IDs a record (or set of records) and value stores exact location on disk. Then, instead of having to pull large and potentially multiple blocks of data to RAM, before data found, know exactly where it is, and pull just that data, much quicker.

-Indexing is like index in a book. If try and find chapter in book, may have to flip through whole book to find it. But if use book index, then can just look up exact location, via index, and flip right to correct chapter.

-Dense Index - index maintains a key-value mapping for all records in DB

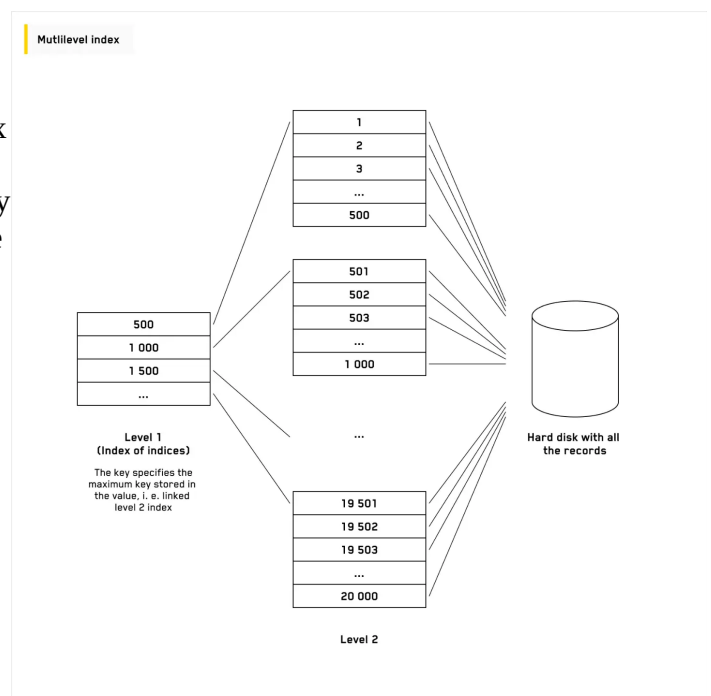
-Sparse Index - Only subsets of records are put into index

-When use index, load index into RAM, and search index for location, then pull data from location, versus without index, where load actual DB data blocks into RAM, then search blocks for data.

-If DB has hundreds of blocks, could have to search $O(n)$ blocks to get data. With indexing, just need $O(1)$ actions, one to load index into RAM, then one to pull data from exact location, per index.

-Indexing downside is slower writes, even if faster read, as each write must also update the index. For write heavy systems, be careful with indexes, as such. Monitor queries and add indexes based on actual usage patterns, not assumptions.

-Multilevel Index - As DB size grows, index size also grows and may require multiple I/O calls to read full index. Multilevel indexes contain an index of indices, where to get DB data, first call index of indices, to get desired indexes, then get specific key from that specific index, to get block data. Do have to cycle through each set of indexes one by one, until find desired specific index, but still much faster than cycling through *all* blocks until find desired index.



B trees

$k1 < k2 < k3$

Node with keys < k1

Node with $k1 < \text{keys} < k2$

Node with $k2 < \text{keys} < k3$

Node with keys > k3

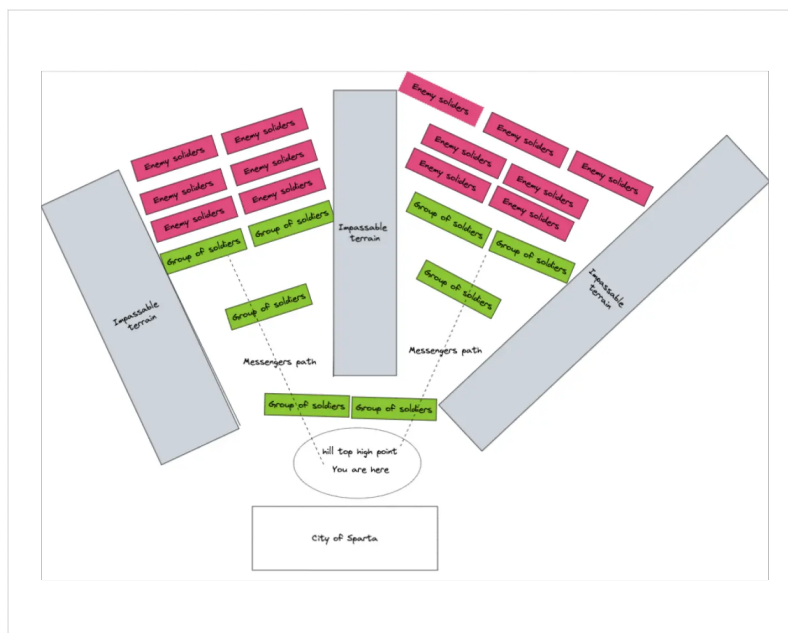
$k1, k2, k3$ represent the keys
 $r1, r2, r3$ are the reference to the corresponding records

X) Failover

-Complications of failover:

- Whenever failover occurs, leader must be fully be demoted, with the leader itself doing zero more leader tasks, and instead become follower. Meaning, stop accepting write requests, and instead be configured to listen to new leader.

- Making multiple copies of the same data on different machines



-Replications Uses:

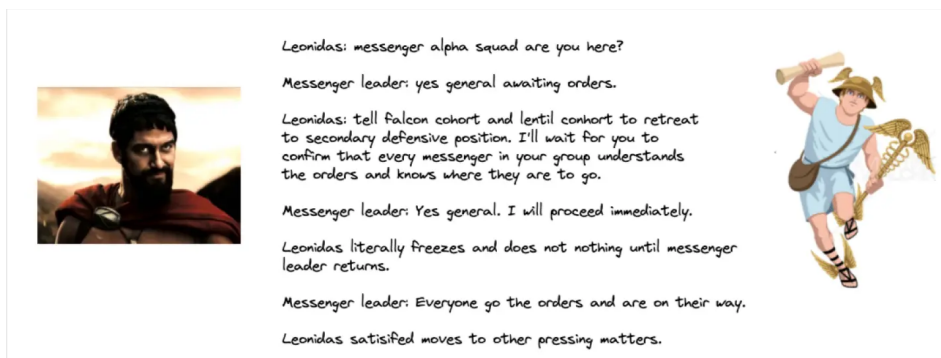
- To provide a fallback and prevent single point of failure in case machine goes down
- To better serve global users by serving them a copy of data close to them
- To increase throughput to allow serving of more requests via more machines

-Replica - copy of data

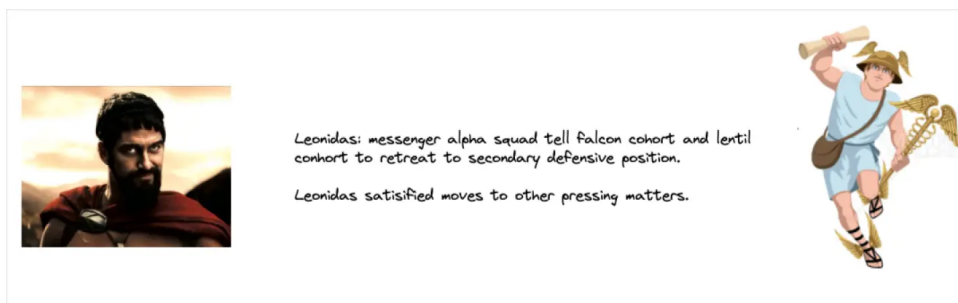
-Leader - machine that handles write requests to data store

-Followers - replicas of the leader node. Handle read requests.

-Synchronous Replication - Replication that is complete only when write requests to all replicas are also marked as acknowledged. Leader must wait for acknowledgment of "processed" from all followers for replication to become synchronous.



-Asynchronous Replication - Replication that is complete even when all write requests to all followers have not responded as acknowledged.

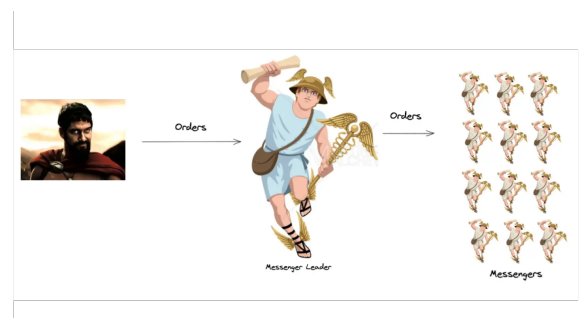


-Sync Replication requires sync to finish before moving on. Async can move on before done. Sync thus has better data consistency expectations, but also can be slower. Async has lower latency for writes, but comes with increased risk of data inconsistency during failure scenarios.

-Hybrid Sync and Async Replication can be used, where replication is considered complete when one sync follower completes replication, even if other followers (all async) are not yet done with replication. Makes sure at least one complete copy, at all times.

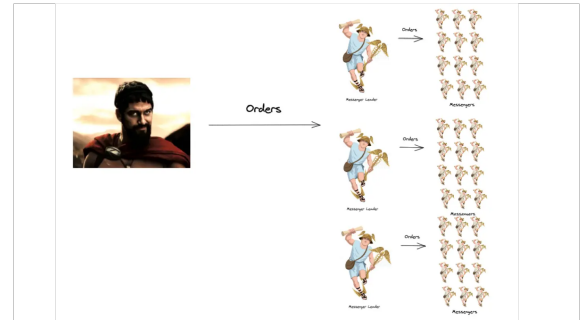
-Types of Replication:

- Single Leader
 - Single machine as leader (performing all data store updates) while all other machines handle read requests.

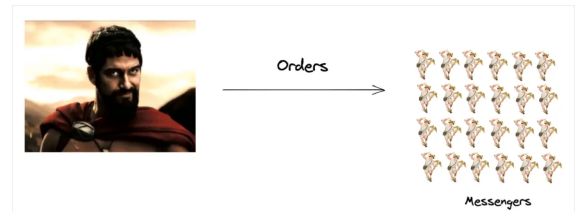


- Leader also passes down write info to followers, so they have up-to-date data too.
- If leader fails, follower becomes new leader. AKA "master-slave," "primary-standby," or "active-passive" replication.

- Multi Leader
 - Same as single leader, but multiple leaders, each with their own separate set of followers.
 - This allows more machines to handle writes.
 - It also means more leaders, so more writes to leaders needed too.



- Leaderless
 - All machine handle read and writes.
 - Sometimes client directly writes to all machines.
 - In other cases, client contacts coordinator, that broadcasts request to all nodes.



XII) Consistent Hashing

-A way to make sure we are still balancing distributed system load properly, despite system servers being added and removed for a variety of unpredictable reasons.

-A way to effectively distribute the keys in a distributed storage system (distributed cache, sharded DB, etc.) to a large number of nodes or servers, while also allowing to add or remove nodes/servers without incurring a large performance hit.

-If traditional hashed-based data distribution in use, must re-map all items when number of servers changes.

-Consistent hashing using a "ring" structure for hashing. Keys are mapped to nodes/servers. When node/server is added or removed, only a small num of keys (those mapped to the affected server) need to be re-mapped.

-Especially useful for distributed systems, such as content delivery network, as significantly reduces cost of re-sizing a cluster, while also allowing low cost re-sizing, plus ensuring proper balanced and uniform load to distributed systems.

Part III: A 3-step Interview Framework

Intro

Framework Fundamentals

1. Give sys design interview answers in an opinionated method that can be applied to most interview questions. Keep the process fixed, so you can focus on the problem more.
2. The framework's three steps can each be thought of as functions, which take a single input, and return a single output. They can thus be practiced in isolation.
3. The framework is a process. A proper process can trump greater knowledge. Systemically decomposing a problem into a proper architecture is impressive.

Using Framework

-Practice the steps separately, as functions. Steps:

1. Requirements
2. Data types, access patterns, and scale
3. Design

Limitations

-The framework being a prescriptive method, it may still need to adapt or alter to questions for which the process does not fit. Adapt via what works best for you, also.

-Spend as much or as little time as you need on each step, relatives to strengths, weaknesses, and past experience.

Framework Overview

1. Requirements
 - Input: Interviewer problem statement
 - Output: List of functional and non-functional requirements
2. Data Types, API and Scale
 - Inputs:
 - Interviewer problem statement
 - Functional and non-functional requirements
 - Outputs:
 - List of data types we need to store
 - Access patterns for data types
 - Expected scale of system data and requests
3. Design
 - Inputs:
 - Interviewer problem statement
 - Functional and non-functional requirements
 - List of data types we need to store

- Access patterns for data types
- Expected scale of system data and requests
- Outputs:
 - Data storage
 - Microservices

Step 1. Requirements

Intro

- Input: Interviewer problem statement
- Output: List of functional and non-functional requirements
- Sys design interviews are purposefully very open ended. Ask good questions to determine proper requirements.
- If an interviewer provides a random specific detail, make sure to note it, as it is a clue to a requirement.
- Functional requirements - what the core product features and uses cases that the system must support
- Process:
 1. Create a Functional Requirements list title on whiteboard
 2. Ask clarifying questions to determine problem details
 3. Stop asking questions when feel all sys use cases have been gathered
 4. List all requirements you've ID'd and validate w/ interviewer
- Determine only what needs to be built, not how. Do not think of scale yet.
- Step 2 (Clarifying Questions) Process:
 1. ID the main business objects and their relations
 - ex. Twitter: Accounts and Tweets. Accounts-to-Tweet relationship? Account to account relationship?
 - "Account can follow Account," "Account can publish Tweet," "Account can like Tweet," "Account can re-tweet Tweet," "Tweet can reference (re-tweet)"
 2. Delve briefly into props of objects
 - Media is a common part of sys design. Always ask if any of the business objects you have identified can hold media.
 - ex. can Tweet contain media?
 - Consider mutability. Can system object be mutated or can they assumed to be immutable?
 - ex. can Tweets be edited or deleted? Can accounts be deleted? What happens to an account when deleted?

3. Determine data access patterns

-Single most influential part of sys design, as access patterns determine data storage type.

-Access pattern requirement: *Given ObjectA, get all related ObjectB*

-ex. (given Account -> get all followers), (... -> get all Accounts they follow), (...-> get all it's Tweets'), (... -> get a curated set of Tweets for Accounts they follow)

-ex. (given a Tweet -> get all Accounts that liked it), (... -> get all accounts that retweeted it)

-Consider all cross relationships of all related objects ObjectA x ObjectB

-Make sure to offer suggestions for access pattern requirements, but do not force them. "Should we be able to get all Tweets for an account?"

-Any access pattern which may involve manipulating the object (ex. ranking), should get further consideration. Strive for simplicity.

-Simplicity Ex. "Is curated Tweet feed good enough if it is composed of all Tweets by followed Accounts, in chronological order?" Perhaps you need a more complex algorithm, that includes "likes," also?

Functional Requirements Ex. - Design Tik Tok

-Interviewee is not familiar with platform, so asks interviewer to give a high-level overview of what we are looking for. Interviewer gives basic overview of platform.

-If interviewee has more functional requirements questions, he asks interviewer, now.

1. Identify the main objects and their relations

-Objects: Account, Video

-Relations: "Account --posts--> Video," "Account --likes--> Video," "Account --comments on-->Video," "Account --follows--> Account"

2. Delve briefly into props of objects

-Object Props: Account: { username, description }, Video: { description }

-Interviewee asks if objects are mutable. "Can videos or comments be changed after uploading?" "Can account be deleted?" "No and no."

3. Determine data access patterns

-Remember: Given X, return all related Y objects

-Get: "Given Account, get all posted videos," "Given Account, get their feed (videos posted by people they follow)," "Given a Video, get likes/comments"

-Put: "Post a video," "Follow an Account," "Like/comment on a Video"

Functional Requirements Ex. - Design a Code Deployment System

-Interviewer describes a system that should be able to tag a release, then package the release, and deploy it to some servers

1. Identify the main objects and their relations

-Objects: Artifact (a file or set of files that are created during the software development process. They can include source code, test suites, and other types of files that help developers build and test their software.)

2. Delve briefly into props of objects

-Object Props: Artifact { productName, version, commitHash }

-Interviewee asks if Artifact can be mutated after published. "No."

3. Determine data access patterns

-Put: "Publish a code artifact," "Deploy a published artifact to server"

-Delete: "Recall a release"

Non-Functional Requirements

-Quality attributes that specify how the system should perform a certain function.

-Most common NFRs

1. Performance
2. Availability
3. Security

-NFRs define what you should be optimizing for. Not everything can be optimized for and do not over-complicate by assuming 100% ideal optimization possible. Trade-offs.

-Can use functional requirements to define NFRs. Ex. "Don't need to focus on consistency as much, as it is okay if some TikTok users get access to some videos slightly quicker than other users."

-Overspecified NFR == too expensive to be practical. Under-specified == system will not fulfill needed requirements.

NFR: Performance

-The system's ability to respond quickly to user requests.

-Better performance might come at cost of consistency or more complex solution.

-Performance is vital when a lot of synchronous user-facing workflows, all wanting an immediate system response.

-Optimize for the synchronous workflows accessed most frequently. Which defined access patterns would need to be *most* performant? Ex. user feed in Twitter, as immediate response expected and lots of accesses.

NFR: Availability

- How much downtime a service can tolerate.
- Better availability might come at cost of expense.
- If taking downtime would result in financial losses or correctness issues, the system should be highly available. What is the cost of downtime?
- If consistency is highly important, some availability may be sacrificed, as at that point system being unavailable or stale for a short time may be worth the gained greater consistency.
- If in doubt, ask on consistency vs availability as which more important.

NFR: Security

- Protection of the system against outside attacks.
- Remember, not the time to talk about specific solutions, but to gather requirements instead.
- Ex. Leetcode security: all submitted Leetcode answers should execute in isolated sandboxes.
- Is there any special workflow that requires security considerations?

NFR Ex. - Design Tik Tok

- Design for performance (many users making similar requests) and availability (immediate response expected)

NFR Ex. - Design a Code Deployment System

- Design for performance (deploy within 1 hour of publish) and Availability (must be highly available, so can publish at almost any time)
- Do not need to consider security as much, as code for deploys are internal, and thus should be considered trusted.

Step 2. Data Types, API, and Scale

Intro

- What step
- Inputs:
 - Interviewer problem statement
 - Functional and non-functional requirements
- Outputs:
 - List of data types (objects) we need to store
 - Access patterns for data objects
 - Expected scale of system data and requests

-At this stage, still determining system details, ask:

- What data types does the system need to store? //data
- What does the API look like? //API
- What volume of requests must system support? //scale

What data types does the system need to store?

-Further define data types based on problem and defined functional and non-functional requirements (already defined objects and their props, now we define prop types)

-Two main data types:

- Structured data - business objects (accounts, orders, tweets, etc.)
- Media and blobs - images, videos, large binary data such as TAR and ZIP files

-Ex. Twitter

- Structured data - accounts, tweets
- Media and blobs - images and videos in tweets

-Also have unstructured data, which are also business objects, but lack clear structure or are routinely changing

What does API look like?

-Here, write out template API, with endpoint placeholders and pseudo-code showing desired action

-Ex. Twitter API:

```
getTweets:
  GET /{accountID}/tweets?nextPageToken={token}
  returns: Paged list of user's tweets sorted by time created desc.

getFeed:
  GET /{accountID}/feed?nextPageToken={token}
  returns: Paged list of tweets for user's feed

postTweet:
  POST /{accountID}/tweets
  body: content of tweet

retweet:
  POST /{accountID}/retweet
  body: id of tweet that is to be re-tweeted
```

-API should be HTTPS whenever possible (90% of time), so always expect communication will be HTTPS during design

What volume of requests must system support?

-To determine scale. Start with is system read vs write heavy. Which API requests will be called most frequently can help determine this (ex. Twitter will be read heavy).

-If not sure on read vs write heavy, ask interviewer which user actions (routes) will be performed most often.

-Should also ask interviewer if they want "back-of-the-envelope" math for expected reads and writes per minute, to determine estimated storage, or if okay to move on.

-Math should look at (estimated requests per second) * (estimated size of data for endpoint) for reads and writes.

-Refresher: (1,024kb == 1mb), (1,000mb === 1gb), (1,000gb == 1tb)

-Ex. Twitter:

- Reads / min: 100k
- Writes / min: 1k
- Structured data (accounts/tweets): 100kb each
- Media (images, videos): 10mb each
- Reasonable average size of write (just a guess!): 1mb
- Final estimation: (1k writes / min * 1mb each) = 1,000mb per min (1gb per min), as such, we will need about 1gb per min for storage

Step 3. Design

Intro

-How and where step

-Inputs:

- Interviewer problem statement
- Functional and non-functional requirements
- List of data types (objects) we need to store
- Access patterns for data objects
- Expected scale of system data and requests

-Outputs:

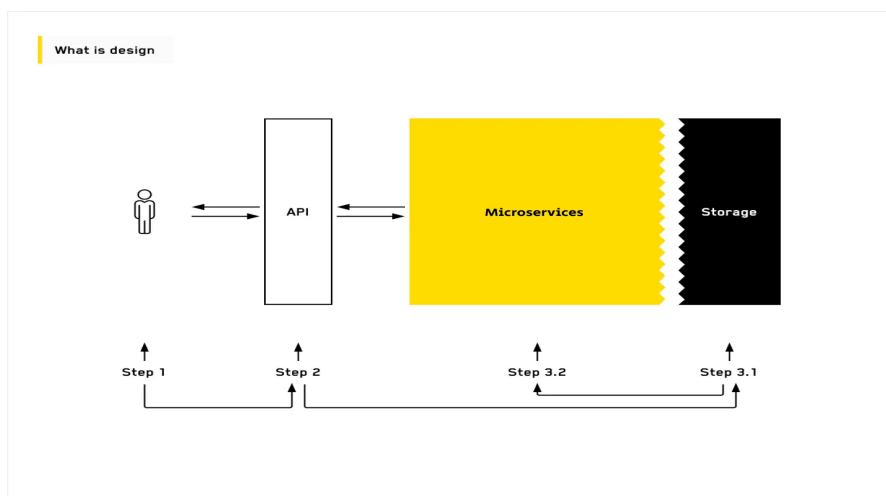
- Data storage
- Infrastructure services

-At this point, have gathered much data. We know uses cases and what to optimize for. Pretty easy to design, now. Want speed? Add cache. Want availability? Add redundant nodes/servers.

-Tell interviewer, "I now have enough requirements and definition for my data, a template API, and required scale. At this point I'll start drawing some WIP design boxes." Then, whiteboard some actual boxes, one for each component in sys.

-Design focuses around two components and their sub-components:

- Data storage - where are we storing data and details
- Infrastructure services - how we store and retrieve data. In between storage and API.



-Design Steps:

1. Data storage layer
2. Infrastructure services

Blob Storage

-Blob storage - Binary large object. Binary data, like ZIP file. Store and retrieve blob as single item.

-If media in data type determination step, blob may be good candidate, so draw box for blob storage.

-ex. Amazon S3, Azure Blob

DB Storage

-Steps:

1. Choose relational vs. non-relational
2. Entities to store

Relational vs. Non-Relational

-Relational vs non is a fundamental step in design. Pick one, say why you are picking that one, and trade-offs vs other choice.

-SQL

- entities need complex relationships and joins, with multi-table relationships
- complex queries, aggregations and reporting
- many multiple table transactions
- very high consistency with strong ACID

-NoSQL

- do not need multi-table relationships and key-value or document storage is sufficient
- simple CRUD operations
- single table transactions
- eventual consistency is acceptable
- easy horizontal scaling

-Relational picked statement: "Although I think relational best matches our requirements, we should be mindful of the downsides. For example, our DB will be more rigid in schema and structure, so it might be harder to change. Additionally, if we horizontally scale via sharding, or so forth, we would need more planning than NoSQL."

-Non-relational picked statement: "Although I think a non-relational DB best matches our requirements, we should be mindful of the downsides. For example, we will not having a locking DB, so we would have to handle potential for write collisions, plus we loose strong ACID guarantees and have less reliable consistency overall."

-Ex. Twitter:

- Relational or non entities? Could be thought of as either. Account table links to Tweets SQL table or Account contains tweets nested in NoSQL.
- Strong consistency needed? Not immediately as okay if takes a moment for tweet, likes, or follows to show for all users. Eventual consistency okay.
- Large amounts of unstructured data? No, as entities relate clearly and have well defined data attr that likely will not change much over time
- As such, given above, could use relational or non-relational, and what more matters is justification. Twitter actually started with SQL, then moved to NoSQL seeking better availability and scalability.

-Ex. Banking System

- Need high ACID and consistency and have structured data --> relational DB

-Ex. App to help doctors diagnose illnesses given symptoms

- Mostly a read system. Illnesses, symptoms, and diagnoses themselves are ever changing, so data thus more unstructured. Will need a lot of scaling as always a lot of new data due to med research. --> non-relational DB

-Ex. Amazon

- Need consistency for orders but still flexibility for data in product catalog. --> Combo. Relational for purchases and stock. Non-relational for product catalog.

Entities to Store

-Choosing how to store entities is an iterative process: identify, implement, assess, improve

-Process:

1. Build a table for each entity with table name and field names
2. Review access pattern and see what table relations they reveal. Add more tables if required.
3. Review which API endpoints (access patterns) will be called most and add indexes
4. See how all access patterns work with current setup. Too slow anywhere?
5. Adjust design and keep iterating it, as required

-Twitter Ex.

1. Build a table for each entity with table name and field names:

- Accounts: id (pkey), name, surname, username

- Tweets: id (pkey), content, author_id, media_url (null allowed)

- Note that null is allowed for media_url as media not required in tweet

2. Review API endpoints and add more tables or fields if needed:

- getTweets: author_id works as pkey

- getFeed: don't have table for users and who they follow yet...need to add this

- putTweet: this is fine as is

- reTweet: this is fine as is

- new table created for feed: account_id, following_id

- Now to getFeed, can query, "Given account_id, following_id" and then given following_id, query tweets table for all tweets by list of followers

3. Review frequent queries and add indexes:

- As determined read heavy, makes sense to add index on "get tweets by author":

 - CREATE INDEX tweets_by_author_id ON tweets(author_id);*

- To help speed up getFollowing, also decide to add index on getting all following for given acct:

 - CREATE INDEX following_by_author on feed(author_id);*

- Remember indexes speed up reads, but slow down writes, so be selective.

4. Review current design:

- For getFeed, first need to get all following_id using author_id from feed table, then get all tweets for all following_id. Is this too slow? Seems expensive.

- Ask interviewer on this: "Although this works from a functional perspective, I'm worried this might be too computationally expensive and slow down reads. Do you think this would be a good place to optimize or would you like to leave this for now?"

5. Adjust & iterate:

- If interviewer suggests optimization, suggest adding cache for user, or storing results in a new table, for faster future searching:

 - feed_tweets(author_id, following_id, tweet_id)

- as well as index on author_id:

 - CREATE INDEX followed_tweets_by_author on feed_tweets(author_id);*

- then also update that table as new tweets are posted

-Get creative too:

-Do we need to keep all feed_tweets up to date for all users at all times? Perhaps only add rows to frequent login users, but for infrequent, compute feed on demand.

-All queries that "get all" should also be behind logic that pages the response, to prevent user experienced slowdown. Similar to lazy loading on front end. Ex. Twitter loads tweets page by page, even if scroll seems infinite.

-Make sure to ask interviewer their thoughts during this, frequently. Do not move forward from this step without input from interviewer.

Infrastructure services

-Once data store is defined, move on to defining infrastructure services, which are often implemented via microservices.

-Common infrastructure services:

- Caching
- Load Balancers
- Queuing Systems

Caching

-Consider when computing result is costly, once computed result tends not to change very often (or at all), and with frequently read objects

-Caching ex. - Redis, which caches parts of DB in memory for faster access

-Downside of cache means adding replication of data which may result in consistency issues if cache falls out of date. Also costly and adds complexity, as updates to DB require cache updates, too.

-Not all systems need caching. Use only if requirements define it's necessity.

-For bonus points, discuss cache invalidation strategies mentioned above, like LRU

Load Balancing

-Horizontal load balancing easier when API servers are stateless, as requests can go to any node. As long as API servers are stateless, they just act as a simple interface to query infrastructure.

-Twitter ex.

- Add replica nodes for API, which allows more servers to handle more load (horizontal scaling), and which creates high availability, as if one server fails, another can take over or be spun up.

Queuing Systems

-Typically done via message queues, which temporarily store messages between producers and consumers in async systems

-Queues make distributed systems more efficient by preventing systems from being overwhelmed by too many requests, while still eventually fulfilling all requests

-Async Systems enable "fire and forget" calls where client sends message to queue instead of waiting for immediate processing

-Queuing ex. - Kafka, RabbitMQ, or Amazon SQS for handling expensive operations and decoupling systems

-Downside of queues means adding complexity and potential message ordering issues. Also add in extra infrastructure management and monitoring

-Not all systems need queuing. Use only if requirements define high throughput or async processing necessity.

3-step Framework End-to-End Example

Step 1. Requirements

-Interview Question: Design interviewing.io, a site where users can schedule anonymous technical interviews with other engineers.

1. Candidate creates "**Functional Requirements**" list on whiteboard, then IDs need for two objects, users and interviews. Writes these down. Asks if need to save recording of interview, too.
2. Interviewer: Yes. We will also need a "showcase" feature, so others can view the interview (if both parties of interview agree).
3. Candidate: Confirms and adds to FR list. Asks if other props about interview they should know about.
4. Interviewer: States interviews only done in one lang and should record that, so can filter "showcase" by lang.
5. Candidate: Confirms and adds to FR list. Asks how interviewers and matched with interviewees.
6. Interviewer: States is based on interviewer availability, which interviewer sets.
7. Candidate: Confirms and adds to FR list. Asks if we need to store code answers, or just recordings.
8. Interviewer: States just storing recordings is fine, but code will need to execute and print from within the interviewing.io service.
9. Candidate: Confirms and adds to FR list.

Functional Requirements

-Problem: Site where users can schedule anon tech interviews w/ other eng

-Business Object: User

-Business Object: Interview

-Feature: Showcase

- Feature: Filter showcase by language of interview
- Feature: Interviewer sets availability
- Feature: Interviewee matched w/ interviewer based on availabilities
- Business Object: Bookings
- Business Object: CodeExecutor

10. Candidate now has enough info to sketch out business objects on whiteboard:

Business Objects

- User: { firstName, lastName, userName, availability }
- Interview: { interviewer, interviewee, videoRecording, progLang, isShowcase }
- Bookings: { timestamp, interviewer, interviewee }
- CodeExecutor: must execute code, run tests on, and display output in window

11. Now defining business objects, candidate can define their relationships on whiteboard:

Business Object Relationships

- User gets all of their recorded interviews (User x Interview)
- User gets all showcase interviews (User x Interview)
- User gets all showcase interviews by programming lang of Interview (User x Interview)
- User gets all scheduled interviews (User x Bookings)
- User sets availability (User)
- User books available interviewer from Bookings (User x Bookings)
- User joins interview (User x Interview)
- Interviewer submits video recording and prog lang for completed interview (User x Interview)
- Interviewee executes their code answer (User x CodeExecutor)

12. Candidate now considers **non-functional** requirements via whiteboard list:

Non-Functional Requirements

- High availability, to ensure reliable user experience during interviews
- Good audio and video streaming quality
- Isolated code execution environments
- Proper bandwidth for re-playing recorded interviews (including showcase)

13. Candidate asks interviewer if this all looks good and if they feel like we are ready to move on. Interviewer confirms, so candidate moves on.

Step 2. Data Types, API, and Scale

1. Candidate now fleshes out business objects via whiteboard lists of structured data and media:

Structured Data

- User: { id, first_name, last_name, username, availability }
- Interview: { id, interviewer_id, interviewee_id, video_recording, prog_lang, is_showcase }
- Bookings: { id, interviewer_id, interviewee_id, timestamp }

Media

- Interview Recording: video

2. Candidate moves on to defining basic REST API interface (using defined business relationships):

```
putAvailability:
  POST /users/{userId}/availability

getInterviews:
  GET /users/{userId}/interviews
  returns: list of interview_id and metadata
getInterview:
  GET /interviews/{interviewId}
  returns: streaming interview

getShowcase:
  GET /showcase{showcaseID}
  returns: streaming showcase interview
getShowcasesByLang:
  GET /showcases/{prog_lang}/langShowcases
  returns: list of showcase interview_id and metadata

getBookings:
  GET /users/{userID}/bookings|
  returns: list of bookings_ids
putBooking:
  POST /users/{userId}/bookings

joinInterview:
  POST /interviews/{bookingId}
  returns: link to coder pad and streaming UI

submitRecording:
  POST /interviews/{interviewId}
```

3. Candidate now moves on to scale, starting with clarifying questions, and a whiteboard list for scale notes

- How many interviewers are we expecting?
- How long does interview last?
- What do we need to store as part of interview result?

Scale

- A thousand interviews per day
- Interviews last about 1 hour
- Just the video recording of the interview

4. Candidate also does some back-of-the-envelope scale math:

1,000 interviews x 1GB per recording = ~1TB of data per day storage required
1,000 live interviews per day * 3 participants (avg) * 1GB streaming = 10TB live bandwidth
10,000 showcase played per day * 1GB per recording = 10TB showcase bandwidth

Step 3. Design

1. Candidate finally moves onto design, starting with data storage:

For storing our structured data, we have quite well defined schema and entities which have tight relations, including relations between both User and Interview, as well as User and Bookings. We also would want our bookings to update transactionally, so we don't have double bookings for interviews. Having said that, SQL makes sense to me here.

User: { id (pkey), first_name, last_name, username, password, email, availability (null allowed) }

Interview: { id (pkey), interviewer_id (fkey), interviewee_id (fkey), video_recording, prog_lang, is_showcase }

Bookings: { id (pkey), interviewer_id (fkey), interviewee_id (fkey), timestamp }

Note, that we salt and hash our use password prior to storing it. Note also, availability can be null, as this is allowed until user sets their availability.

For the recordings, single they are large video files, a blob store makes sense, and the Interview table itself can link to a local file.

2. Candidate reviews their API endpoints and does not find any missing tables or fields
3. Candidate reviews API endpoints to see which might require indexing or additional consideration.

Decides to add index on both showcase interviews:

```
CREATE INDEX showcase ON interview(showcase);  
CREATE INDEX showcase_by_lang ON interview(showcase, lang);
```

Also states that both of these showcase API calls should return paginated data.

4. Candidate then stops and reviews current design.

Considers indexes on showcase. How often would we be updating these fields? Not often, so don't have to worry about write impact as much.

Everything else seems good, so moves on

5. Candidate then asks interviewer for input on the current design, and if the interviewer feels we could use a caching layer here.

Interviewer states the design looks good, but caching for some video recordings be a good idea.

Candidate decides to follow the 80/20 rule and add a CDN caching layer for the the top watched showcase videos, with cache invalidation handled by LRU pattern. Since videos will not be updated after initially uploaded, there is very little "stale data" risk here, despite the downside

of added complexity, too.

6. Candidate asks if additional iteration is needed and interviewer brings up fact that there is no scaling or fallback plan included, and asks candidate's thoughts on system's need for a messaging queue at any point.

Candidate also considers load balancing and need for high availability. Decides to implement horizontal scaling, as the APIs required for this would work with with microservices, which scale horizontally with easy, using least connections / response time for load balancing, as this works well for video streaming. Failover can also be handled via horizontal scaling, as if one machine goes down, others can be brought up. Adds DB backups too, for additional data failover, via single leader DB replication.

Candidate also considers messaging queues, which could be useful for uploading many videos. With only 1,000 interviews and 10k showcases a day, plus horizontal scaling, to start, message queues do not seem needed at this time, and do not want to break YANGI. Waiting on async uploads could also worsen interviewee experience. Still, as number of interviews per day increases, would likely be a good idea to add a queue to handle uploading/processing of many recordings, async.

7. Candidate asks for interviewee input again and interviewer states this looks good now, and sys design interview thus ends (well).

Part IV: Examples

-Note: The examples in this section of notes are brief, often diverge from the previously discussed framework steps (ex. in the first example, it completely skips the "Data Types, API, and Scale" step and goes right into the "Design" step), and sometimes presents conflicting information compared to the previous chapters. It also presents tons of new info not previously discussed.

-I thus skipped taking notes on this section. This article was written by a team and at times it feels like the team members did not align on how they understood the framework or what it contained, at times. This chapter is a good example of that.

-Instead of putting too much time into this section, I did 4 practice system design interview questions with Claude AI 4.1 Opus and Faang+ interviewers through paid mock sys design interview sessions.

-If you still want to read this section, it is at:

<https://interviewing.io/guides/system-design-interview/part-four>